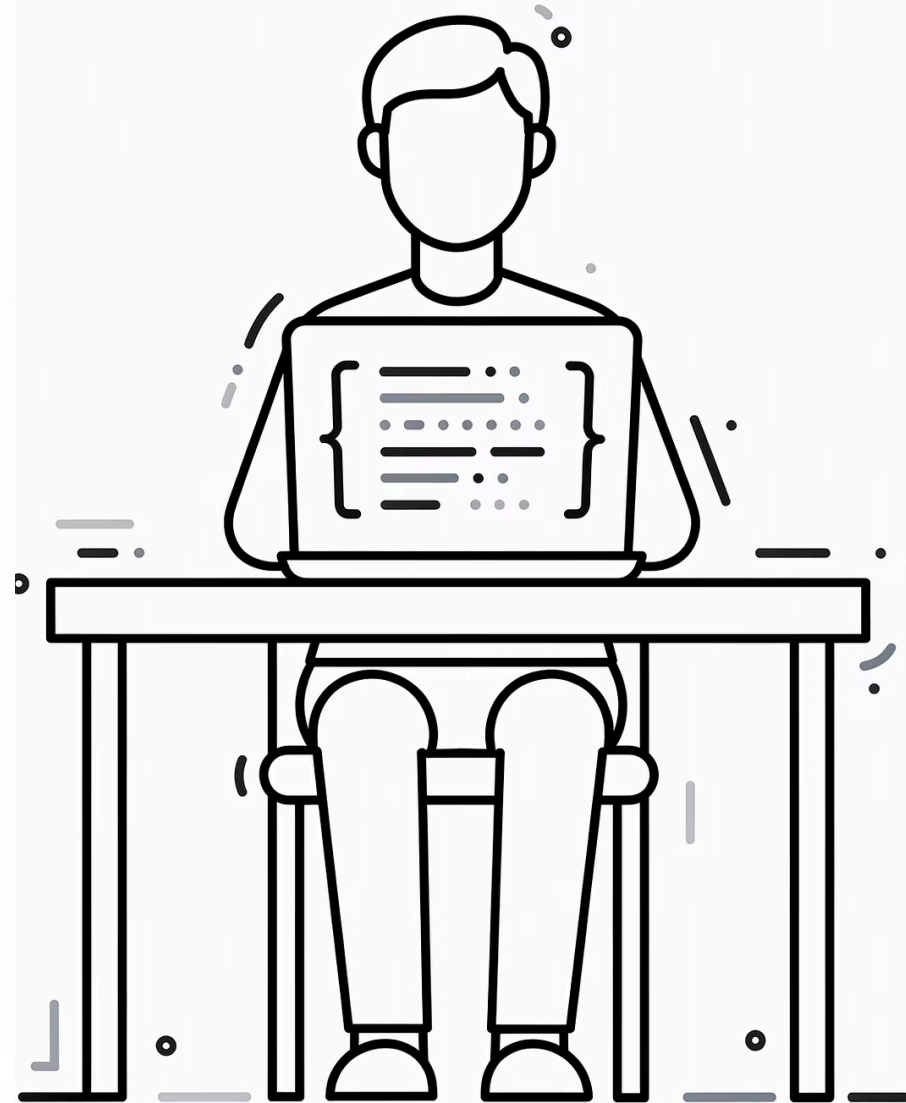


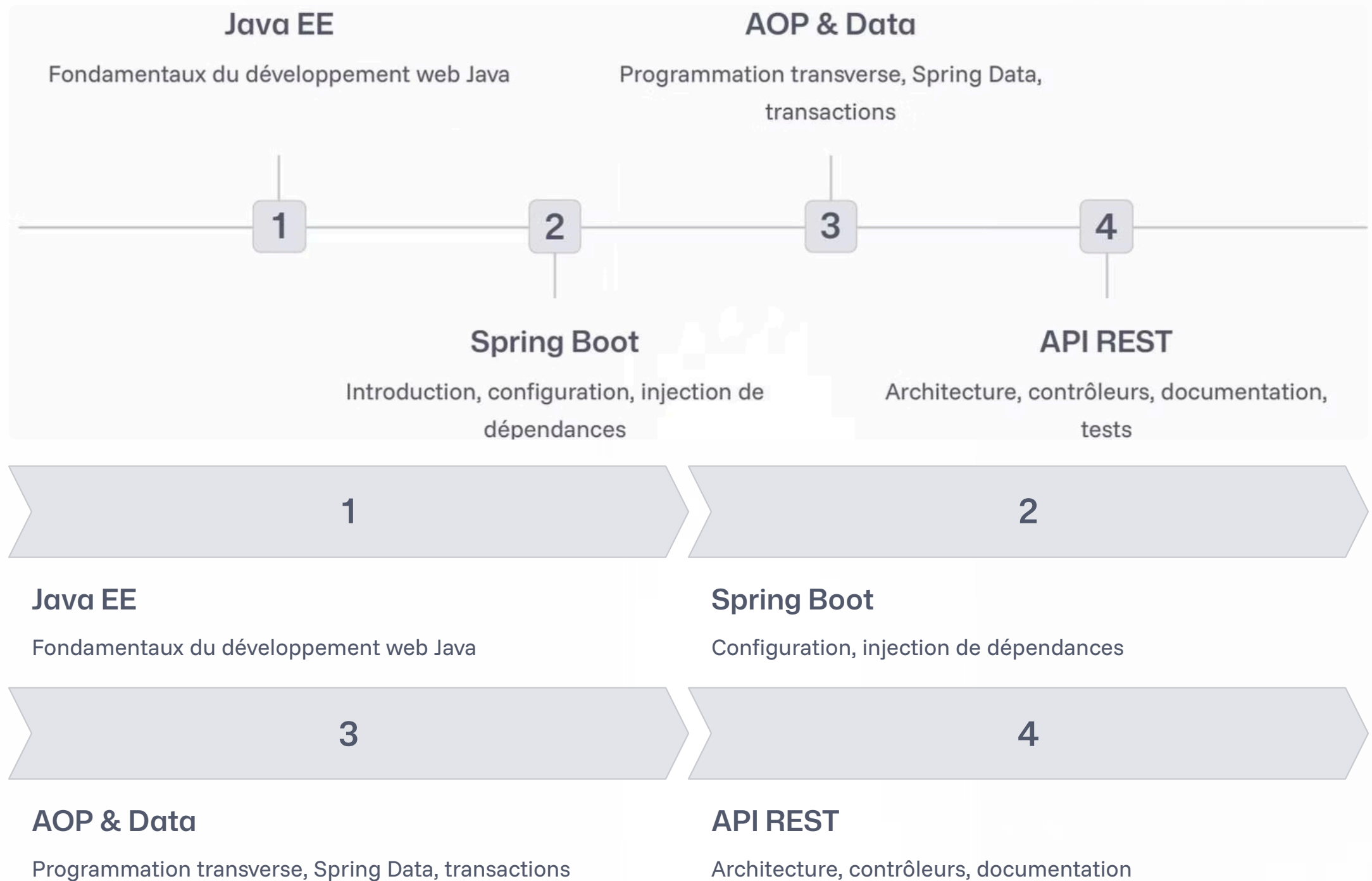
Spring MVC REST

Construction d'API REST robustes avec Spring MVC — architecture, contrôleurs, documentation OpenAPI et bonnes pratiques d'entreprise.



Parcours de la formation

Ce module s'inscrit dans un cursus progressif en quatre étapes.



Objectifs du module

À l'issue de ce module, vous serez en mesure de concevoir, implémenter et documenter des API REST de qualité professionnelle.



Concevoir

Modéliser une API conforme aux principes RESTful



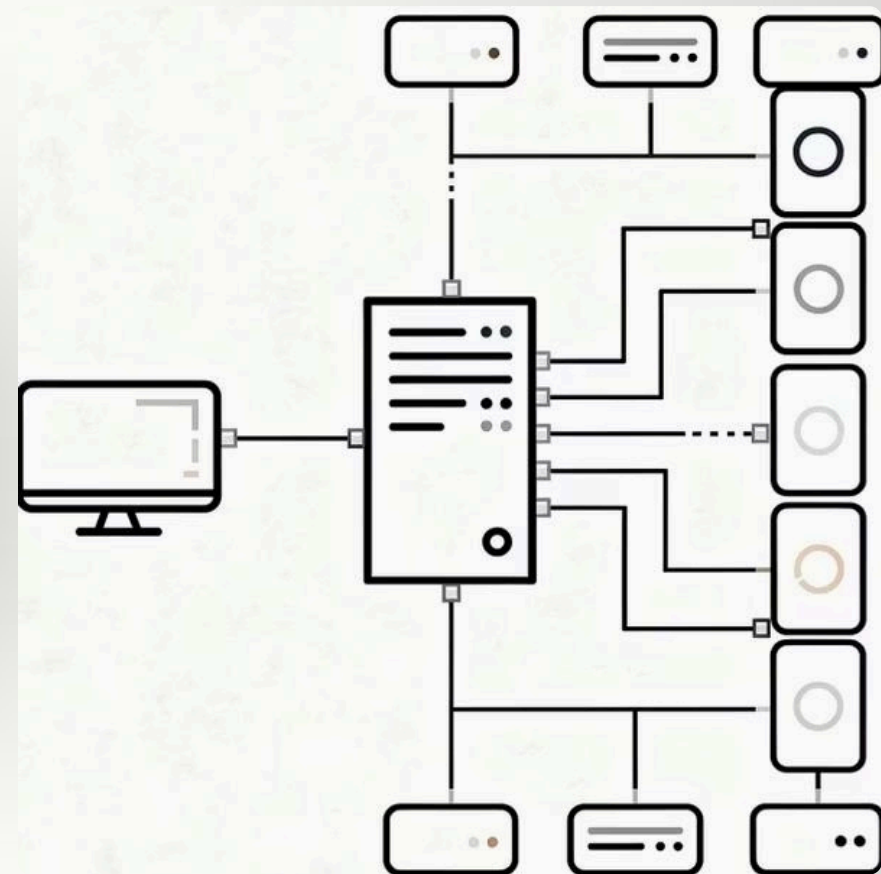
Implémenter

Développer des contrôleurs robustes avec validation et gestion des erreurs



Documenter

Générer et publier une documentation OpenAPI/Swagger



CHAPITRE 1

Architecture REST avec Spring MVC

Comprendre les principes sous-jacents est indispensable avant d'écrire la moindre ligne de code de contrôleur.

Qu'est-ce que REST ?

Définition

REST (*Representational State Transfer*) est un style architectural défini par Roy Fielding en 2000. Ce n'est pas un protocole, mais un ensemble de contraintes conférant scalabilité, performance et évolutivité.

Les 4 contraintes clés

Client-Server

Séparation claire des responsabilités

Stateless

Chaque requête est autonome, sans état de session

Cache

Les réponses indiquent leur cachabilité

Interface uniforme

Identification des ressources, HATEOAS

Les niveaux de maturité REST

Le modèle de Richardson identifie quatre niveaux. La grande majorité des API d'entreprise visent le **niveau 2** — une cible professionnelle tout à fait acceptable.

1

Niveau 0 – Marais du POX

Un seul endpoint, tout passe par POST

2

Niveau 1 – Ressources

URI distincts par ressource, un seul verbe HTTP

3

Niveau 2 – Verbes HTTP ✓

GET, POST, PUT, DELETE et codes de statut corrects

4

Niveau 3 – HATEOAS

Réponses avec liens vers les actions possibles

Spring MVC : du MVC classique au REST

Architecture MVC classique

Modèle

Données et logique métier

Vue

Templates HTML
(Thymeleaf, JSP)

Contrôleur

Coordination des flux

Architecture REST avec Spring MVC

Modèle

Entités et DTOs

Vue

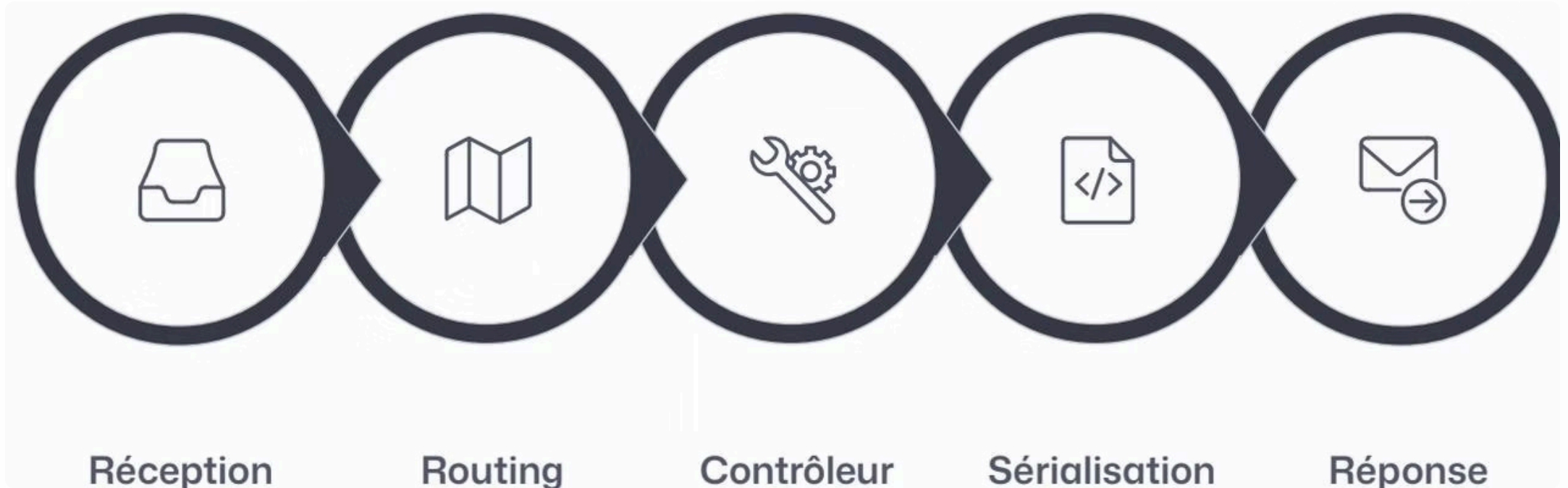
Sérialisation JSON/XML
automatique

Contrôleur

Traitement des requêtes HTTP

 Le `DispatcherServlet` reste le point d'entrée central, déléguant aux contrôleurs via le `HandlerMapping`.

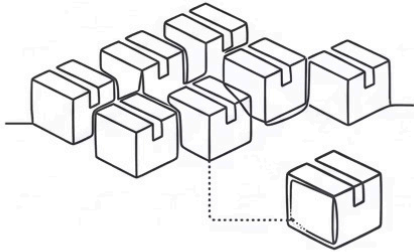
Cycle de vie d'une requête Spring MVC



Le `DispatcherServlet` agit comme un *Front Controller* unique. Il consulte les `HandlerMapping`, invoque la méthode appropriée, puis délègue la sérialisation à un `HttpMessageConverter`. Ce mécanisme est entièrement automatisé par Spring Boot.

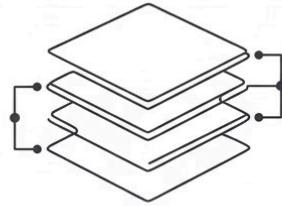
Conception des URI REST

Noms Pluriels



/api/v1/produits
/api/v1/produits/{id}

Hiérarchie de Relations



/api/v1/commandes/
/api/v1/commandes/{id_commande}/articles

Verbes HTTP Standards



Filtrage et Pagination



/api/v1/produits?catégorie=électronique
?page=0&taille=20

Principes fondamentaux

De bons URI sont **intuitifs, hiérarchiques, orientés ressources** (noms, pas verbes) et cohérents.

i Les verbes HTTP expriment l'action — l'URI identifie uniquement la ressource.

Content Negotiation

Négociation côté client

Le client précise ses préférences via l'en-tête `Accept` :

```
Accept: application/json
```

```
Accept: application/xml
```

```
Accept: application/json, application/xml;q=0.9
```

Spring MVC parcourt les `HttpMessageConverter` enregistrés pour produire le type demandé.

Configuration Spring Boot

Jackson est auto-configuré pour JSON. Pour XML, ajouter `jackson-dataformat-xml`. La configuration se personnalise dans un `WebMvcConfigurer`.

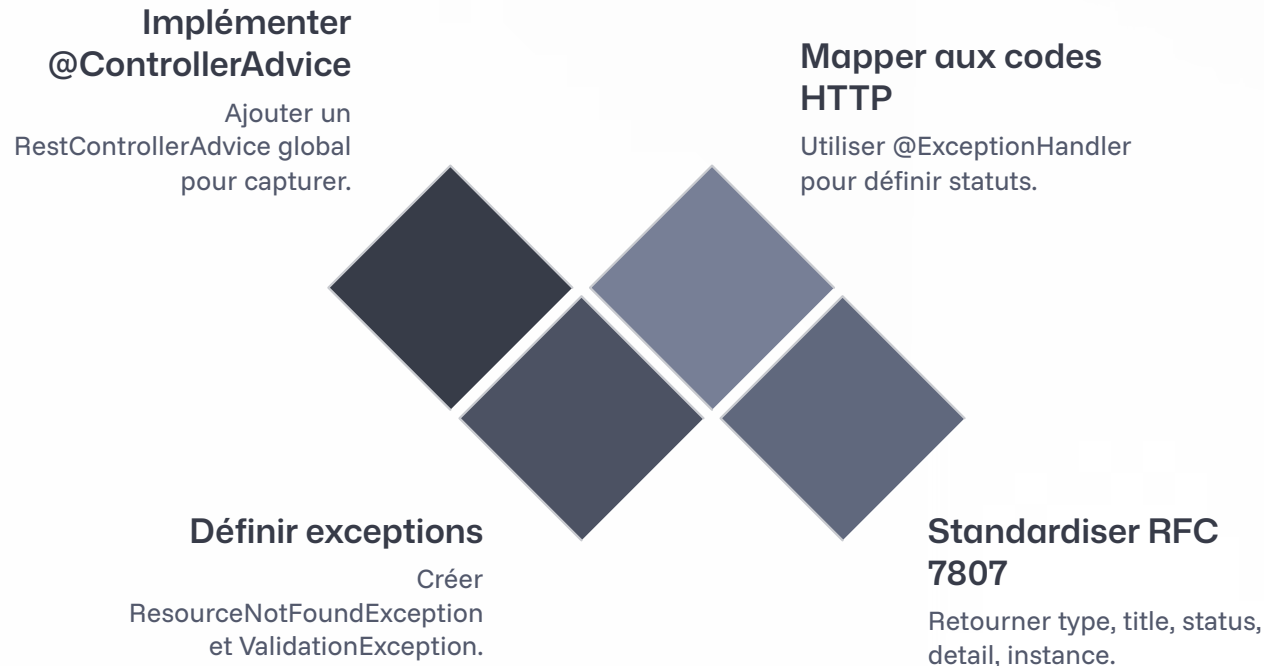
- ✅ La content negotiation permet d'exposer la même ressource en JSON, XML ou CSV sans dupliquer la logique métier.

Codes de statut HTTP

L'utilisation correcte des codes de statut est une marque de professionnalisme. Retourner 200 OK sur une erreur est un anti-pattern à éviter absolument.

Code	Famille	Usage
200 OK	2xx Succès	GET, PUT, PATCH réussis
201 Created	2xx Succès	POST — inclure l'en-tête Location
204 No Content	2xx Succès	DELETE, PUT sans corps
400 Bad Request	4xx Client	Requête malformée, validation échouée
404 Not Found	4xx Client	Ressource inexistante
409 Conflict	4xx Client	Duplication d'une ressource unique
422 Unprocessable	4xx Client	Validation sémantique échouée
500 Internal Error	5xx Serveur	Erreur interne — ne jamais exposer les détails

Gestion centralisée des erreurs



L'approche recommandée

Un `@RestControllerAdvice` global intercepte toutes les exceptions et les transforme en réponses HTTP structurées, conformément au standard **RFC 7807** (Problem Details).

- ⊗ Ne jamais exposer les stack traces Java en production. Elles constituent une fuite d'information sensible sur l'architecture interne.

Validation avec Bean Validation

Annotations courantes

@NotNull /
@NotBlank

@Size(min, max)

@Min / @Max

@Email

@Pattern(regex)

@Future / @Past

Fonctionnement

Spring Boot intègre nativement Bean Validation (JSR 380) via Hibernate Validator. L'annotation `@Valid` sur le paramètre de méthode déclenche la validation automatique.

Les erreurs déclenchent une `MethodArgumentNotValidException` que le `@ControllerAdvice` transforme en réponse **400** avec le détail des champs invalides.



Pour des règles complexes, créez des annotations



CHAPITRE 2

Implémentation des contrôleurs Spring MVC

Annotations, mécanismes de binding et bonnes pratiques pour des contrôleurs REST professionnels, maintenables et testables.

@RestController : l'annotation fondamentale

@Controller

Marque la classe comme contrôleur Spring MVC. Les méthodes retournent des noms de vues.

@ResponseBody

Les retours sont sérialisés directement dans le corps HTTP via MessageConverter.

@RestController

Combine les deux. Toutes les méthodes retournent du JSON directement. **Standard REST.**

```
@RestController
@RequestMapping("/api/v1/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(
        ProductService productService) {
        this.productService = productService;
    }
}
```

Annotations de mapping HTTP

Spring MVC fournit des annotations dédiées pour chaque verbe HTTP. Préférez toujours les annotations spécialisées à `@RequestMapping` générique.

Annotation	Verbe HTTP	Usage typique
<code>@GetMapping</code>	GET	Récupérer une ressource ou collection
<code>@PostMapping</code>	POST	Créer une nouvelle ressource
<code>@PutMapping</code>	PUT	Remplacer intégralement une ressource
<code>@PatchMapping</code>	PATCH	Mettre à jour partiellement une ressource
<code>@DeleteMapping</code>	DELETE	Supprimer une ressource

Paramètres de requête : les annotations essentielles

@PathVariable

Extrait une variable de l'URI :

/products/{id} →

@PathVariable Long

id. Pour identifier une ressource spécifique.



@RequestParam

Lit un paramètre de requête : ?

page=0&size=20.

Idéal pour le filtrage et la pagination.



@RequestBody

Déséréalise le corps

JSON vers un objet

Java. Combiné avec

@Valid pour la

validation automatique.



@RequestHeader

Accède à un en-tête HTTP spécifique : token

d'authentification, langue, version d'API.

ResponseEntity : contrôle total de la réponse

```
// Réponse 200 avec corps
ResponseEntity.ok(product)

// Réponse 201 avec Location
ResponseEntity.created(location)
    .body(created)

// Réponse 204 sans corps
ResponseEntity.noContent().build()

// Réponse personnalisée
ResponseEntity.status(HttpStatus.ACCEPTED)
    .header("X-Custom", "value")
    .body(result)
```

Pourquoi utiliser ResponseEntity ?

Retourner directement un objet Java ne permet pas de :

- Définir précisément le code de statut (201, 204...)
- Ajouter des en-têtes personnalisés (Location, ETag...)
- Retourner une réponse vide de manière explicite
- Différencier 200 OK de 202 Accepted

 En contexte d'entreprise, `ResponseEntity` est la norme pour tous les endpoints sauf les plus simples.

Sérialisation Jackson



Configuration globale

Via `application.properties` ou un bean `ObjectMapper` : format des dates, gestion des nulls, nommage des champs.



Annotations de classe

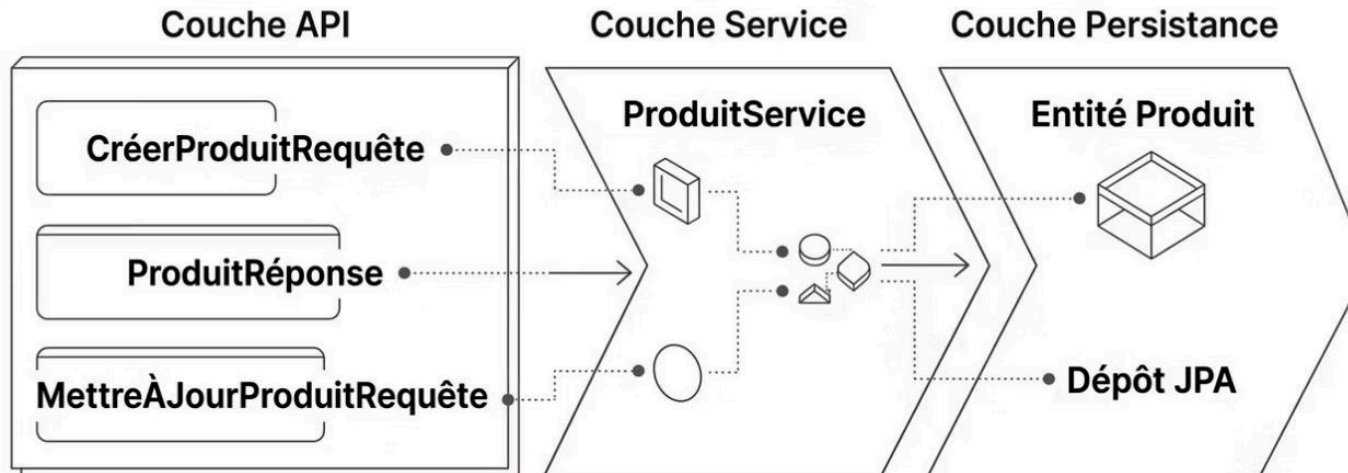
`@JsonProperty`, `@JsonIgnore`, `@JsonFormat`, `@JsonInclude` pour un contrôle précis champ par champ.



DTOs et sécurité

Toujours utiliser des DTOs dédiés. Éviter d'exposer les entités JPA — risque de boucles infinies et de fuite de données.

Pattern DTO



Pourquoi des DTOs ?

Le *Data Transfer Object* sépare le modèle interne de la représentation API :

- Les DTOs de requête valident les données entrantes
- Les DTOs de réponse contrôlent ce qui est exposé
- **MapStruct** automatise la conversion entités ↔ DTOs

✓ Cette séparation apporte flexibilité, sécurité et maintenabilité à long terme.

Un contrôleur REST complet

```
@RestController
@RequestMapping("/api/v1/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService service;

    @GetMapping
    public ResponseEntity<Page<ProductResponse>> findAll(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size) {
        return ResponseEntity.ok(service.findAll(PageRequest.of(page, size)));
    }

    @PostMapping
    public ResponseEntity<ProductResponse> create(
        @Valid @RequestBody CreateProductRequest request) {
        ProductResponse created = service.create(request);
        URI location = URI.create("/api/v1/products/" + created.id());
        return ResponseEntity.created(location).body(created);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

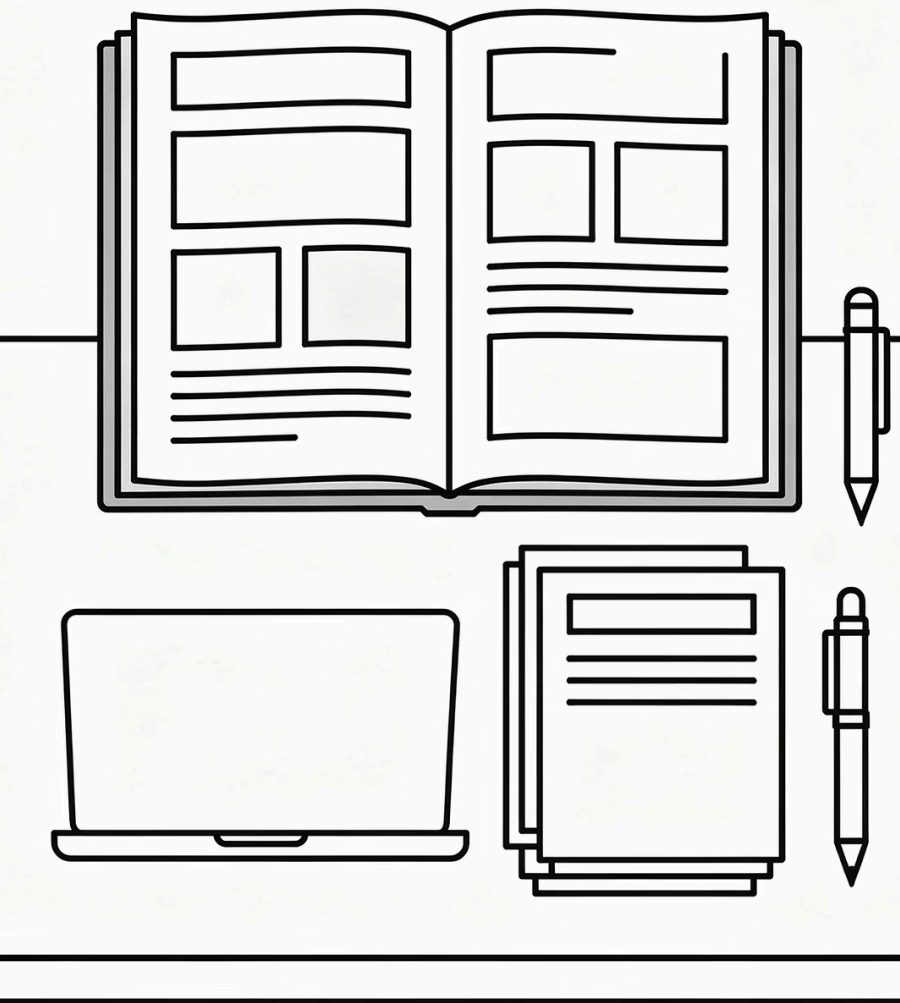
@RestControllerAdvice : gestion globale des erreurs

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
        ProblemDetail pd = ProblemDetail
            .forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Resource Not Found");
        return pd;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setTitle("Validation Failed");
        Map<String, String> errors = ex.getBindingResult()
            .getFieldErrors().stream()
            .collect(toMap(FieldError::getField, FieldError::getDefaultMessage));
        pd.setProperty("violations", errors);
        return pd;
    }
}
```

✔ Spring Boot 3.x introduit nativement `ProblemDetail` (RFC 7807), éliminant le besoin d'une classe d'erreur personnalisée.



CHAPITRE 3

Documentation et versioning d'API

Une API bien conçue n'a de valeur que si elle est correctement documentée. Ce chapitre couvre OpenAPI/Swagger et les stratégies de versioning.

OpenAPI 3 : le standard de documentation



Génération automatique

springdoc-openapi analyse les annotations Spring MVC et génère le fichier JSON/YAML à `/v3/api-docs`



UI interactive

Swagger UI disponible à `/swagger-ui.html` — testez les endpoints directement depuis le navigateur



Contrat d'API

La spec OpenAPI sert de contrat entre équipes frontend/backend, importable dans Postman ou un API Gateway

```
<dependency>
<groupId>org.springdoc</groupId>
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
<version>2.3.0</version>
</dependency>
```

Enrichir la documentation OpenAPI

```
@Operation(  
    summary = "Récupérer un produit",  
    description = "Retourne le détail complet. " +  
        "Retourne 404 si inexistant.")  
@ApiResponses({  
    @ApiResponse(responseCode = "200",  
        content = @Content(schema =  
            @Schema(implementation =  
                ProductResponse.class))),  
    @ApiResponse(responseCode = "404",  
        content = @Content(schema =  
            @Schema(implementation =  
                ProblemDetail.class)))  
})  
@GetMapping("/{id}")  
public ResponseEntity<ProductResponse> findById(  
    @Parameter(description = "ID du produit",  
        example = "42")  
    @PathVariable Long id) {  
    return ResponseEntity.ok(service.findById(id));  
}
```

Annotations clés

@Operation

Résumé et description de l'endpoint

@ApiResponse

Documente chaque code de retour possible

@Parameter

Décrit les paramètres avec exemples

@Schema

Annote les DTOs champ par champ

 Documentez systématiquement tous les codes de réponse, y compris les cas d'erreur.

Configuration globale OpenAPI

Bean de configuration Java

```
@Configuration
public class OpenApiConfig {

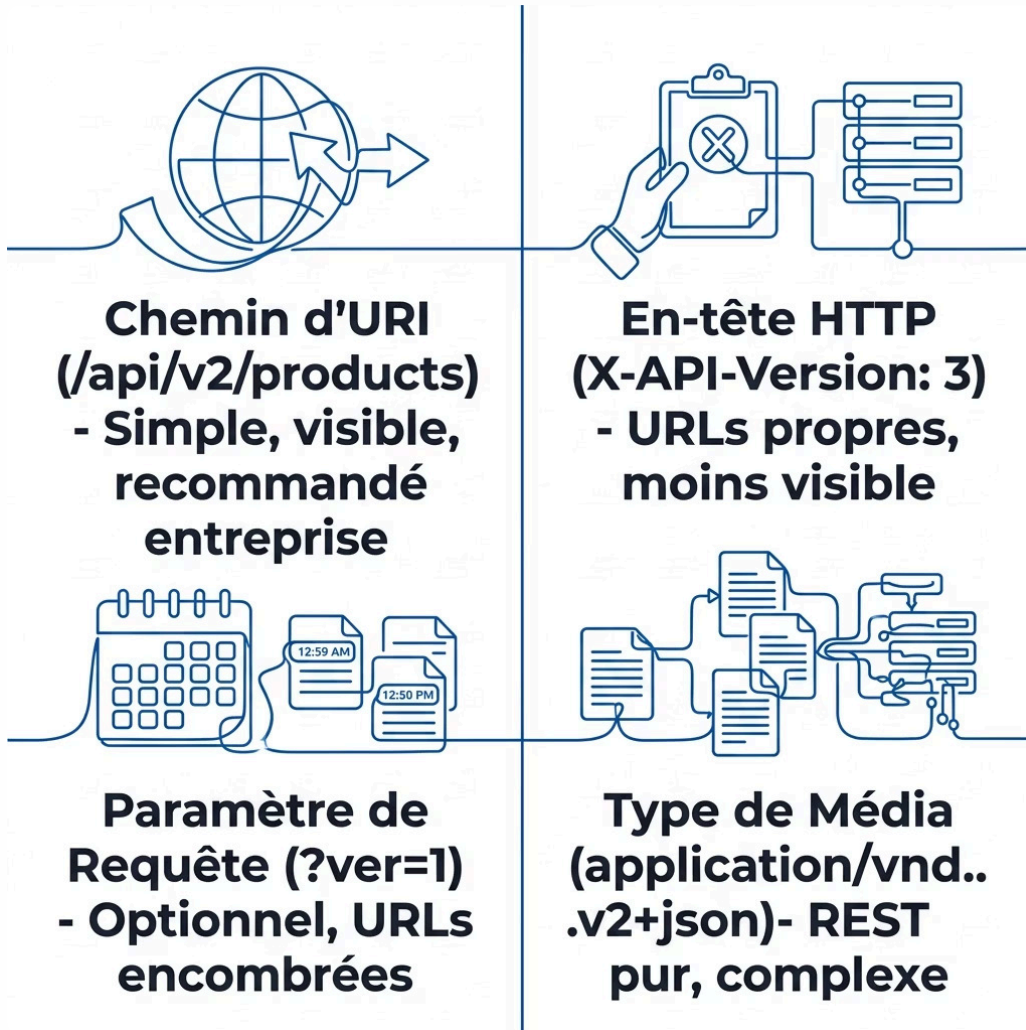
    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Products API")
                .version("1.0.0")
                .contact(new Contact()
                    .name("Équipe Backend")
                    .email("api@company.com")))
            .components(new Components()
                .addSecuritySchemes("bearer-jwt",
                    new SecurityScheme()
                        .type(HTTP)
                        .scheme("bearer")
                        .bearerFormat("JWT")));
    }
}
```

Configuration via properties

```
springdoc.api-docs.path=/v3/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.swagger-ui.operationsSorter=method
springdoc.swagger-ui.tagsSorter=alpha
springdoc.packages-to-scan=com.company.api
```

 La configuration Java est préférable pour les informations riches (sécurité, métadonnées). Les properties conviennent pour les paramètres d'URL.

Versioning d'API : stratégies



Recommandation d'entreprise

Le **versioning par URI** (/api/v1/, /api/v2/) est la stratégie la plus répandue : visible, simple à comprendre et facile à tester.

❌ Choisissez votre stratégie dès le premier jour et tenez-y-vous. Changer en cours de route est extrêmement coûteux.

Implémentation du versioning par URI

Version 1 – Classe dédiée

```
@RestController
@RequestMapping("/api/v1/products")
public class ProductV1Controller {

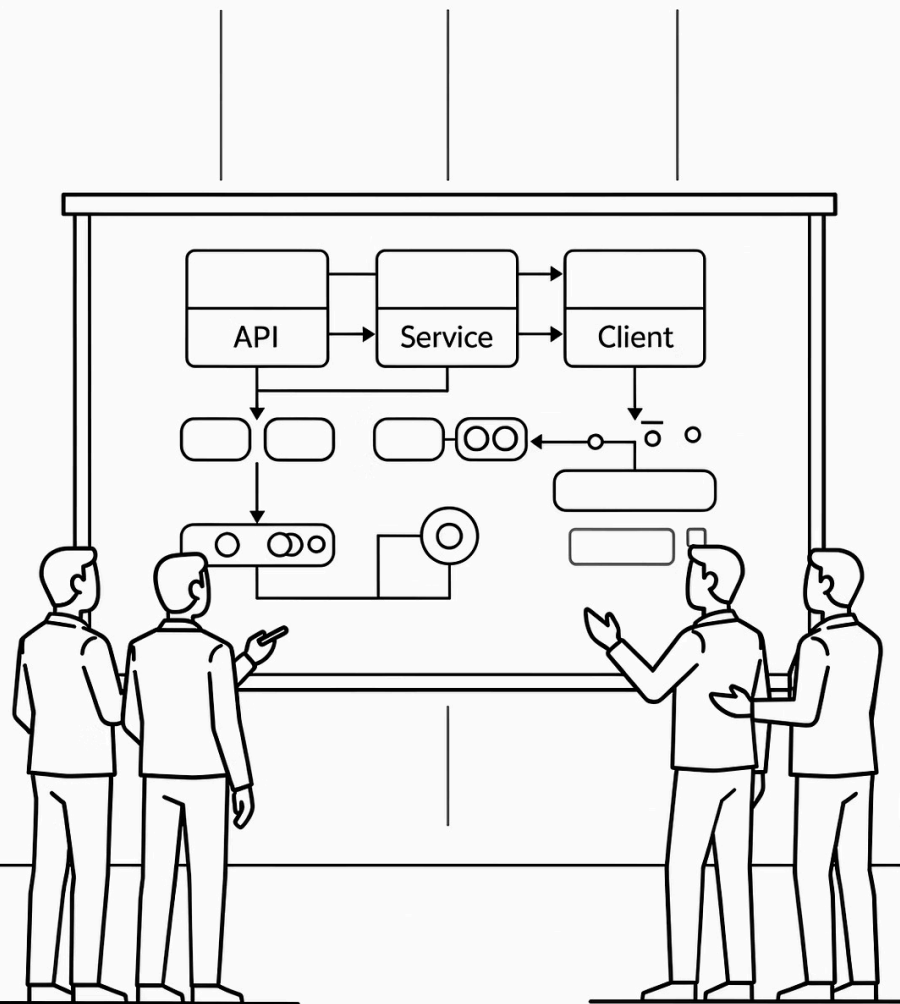
    @GetMapping("/{id}")
    public ResponseEntity<ProductV1Response>
        findById(@PathVariable Long id) {
        return ResponseEntity.ok(
            service.findByIdV1(id));
    }
}
```

Version 2 – Évolution non-breaking

```
@RestController
@RequestMapping("/api/v2/products")
public class ProductV2Controller {

    @GetMapping("/{id}")
    public ResponseEntity<ProductV2Response>
        findById(@PathVariable Long id) {
        // V2Response ajoute de nouveaux champs
        // sans supprimer les anciens
        return ResponseEntity.ok(
            service.findByIdV2(id));
    }
}
```

 Maintenez les anciennes versions le temps que tous les clients migrent. Communiquez les dates de dépréciation via l'en-tête `Sunset` et dans OpenAPI.



BONNES PRATIQUES

Les 10 commandements de l'API REST professionnelle

Commandements 1 à 5

1

Nommer les ressources avec des noms

`/products` ✓ — `/getProducts` ✗. Les verbes HTTP expriment l'action.

2

Utiliser les codes HTTP avec précision

201 pour la création, 204 pour la suppression, 422 pour les erreurs sémantiques.

3

Toujours utiliser des DTOs

Ne jamais exposer les entités JPA directement. Contrôle total sur les données exposées.

4

Valider systématiquement les entrées

Bean Validation sur tous les DTOs de requête. Messages d'erreur clairs sur les champs invalides.

5

Versionner dès le premier jour

Préfixez toujours avec `/api/v1/`. Le coût d'ajout est nul, le coût de rétrofit est énorme.

Commandements 6 à 10

1

Paginer toutes les collections

Ne jamais retourner une collection sans pagination. Page 0, taille 20 comme défaut raisonnable.

2

Documenter les erreurs autant que les succès

Chaque endpoint documente tous ses codes de retour dans OpenAPI, y compris 400, 404, 409 et 500.

3

Sécuriser les endpoints par défaut

Principe du moindre privilège. Les endpoints publics sont des exceptions explicites, pas la règle.

4

Logger les requêtes et réponses

Logs structurés JSON avec correlation ID pour tracer une requête dans tout le système distribué.

5

Tester les cas d'erreur autant que les succès

Ressource inexistante, données invalides, droits insuffisants, erreur serveur simulée.

Anti-patterns courants à éviter

🚫 Verbes dans les URI

`/createProduct`, `/deleteUser`. Ce n'est pas REST, c'est du RPC sur HTTP.

🚫 Exposer les entités JPA

Risque de boucles infinies de sérialisation et de fuite de données sensibles.

🚫 Collections non paginées

`List<Product>` retournant potentiellement des millions d'enregistrements.

🚫 Codes HTTP uniformes

Retourner 200 sur toutes les réponses, y compris les erreurs. Casse tous les outils HTTP standard.

HATEOAS : vers une API hypermedia

```
{
  "id": 42,
  "name": "Laptop Pro",
  "price": 1299.99,
  "status": "IN_STOCK",
  "_links": {
    "self": {
      "href": "/api/v1/products/42"
    },
    "update": {
      "href": "/api/v1/products/42",
      "method": "PUT"
    },
    "delete": {
      "href": "/api/v1/products/42",
      "method": "DELETE"
    },
    "reviews": {
      "href": "/api/v1/products/42/reviews"
    }
  }
}
```

Quand adopter HATEOAS ?

HATEOAS est pertinent pour les **API publiques à durée de vie longue**, exposées à de nombreux clients hétérogènes, ou pour des domaines avec des workflows complexes.

Pour les APIs internes entre microservices, le surcoût de complexité dépasse souvent les bénéfices.

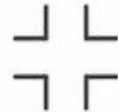
 Spring HATEOAS propose `EntityModel` et `CollectionModel` pour encapsuler les ressources avec leurs liens de manière typée.

Performance des API REST



Éviter le problème N+1

Chaque requête REST ne doit déclencher qu'un nombre limité de requêtes SQL. Utiliser `@EntityGraph` ou `JOIN FETCH`.



Compression des réponses

Activer Gzip/Brotli (`server.compression.enabled=true`) pour réduire la bande passante sur les réponses volumineuses.



Caching HTTP

Exploiter `Cache-Control`, `ETag` et `Last-Modified` pour réduire la charge serveur.



Projections et champs partiels

Support de `?fields=id,name,price` pour éviter de transférer des données inutiles.

Sécurité des API REST

Authentification JWT

Spring Security + JWT : token validé à chaque requête, sans session serveur. Expiration courte + refresh token.

Autorisation fine

`@PreAuthorize("hasAuthority('product:write')")`. Préférer les autorités aux rôles pour une granularité maximale.

CORS configuré

Ne jamais laisser `allowedOrigins("*")` en production. Configurer explicitement origines, méthodes et en-têtes.

Rate Limiting

Protéger les endpoints publics avec Bucket4j ou Resilience4j. Retourner **429 Too Many Requests** avec `Retry-After`.



Sérialisation avancée avec Jackson

Problèmes courants à résoudre

- Dates en ISO 8601 : configurer `JavaTimeModule`
- Exclure les champs null : `@JsonInclude(NON_NULL)`
- Ignorer les champs inconnus :
`@JsonIgnoreProperties(ignoreUnknown=true)`
- Sérialisation des énumérations en chaînes lisibles
- Nommage : `snake_case` vs `camelCase`

Configuration globale recommandée

```
spring:  
  jackson:  
    serialization:  
      write-dates-as-timestamps: false  
    deserialization:  
      fail-on-unknown-properties: false  
      default-property-inclusion: non_null  
      property-naming-strategy: SNAKE_CASE
```



Décidez du naming strategy en début de projet.
Changer en cours de développement impose des modifications côté clients.

Postman : tester les API manuellement



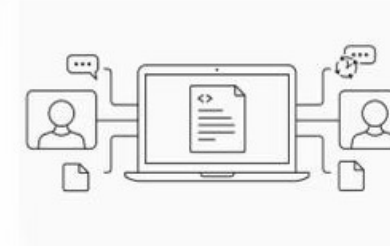
Collections et environnements

Organisez les requêtes par fonctionnalité. Variables d'environnement pour basculer entre local, staging et production.



Tests automatisés

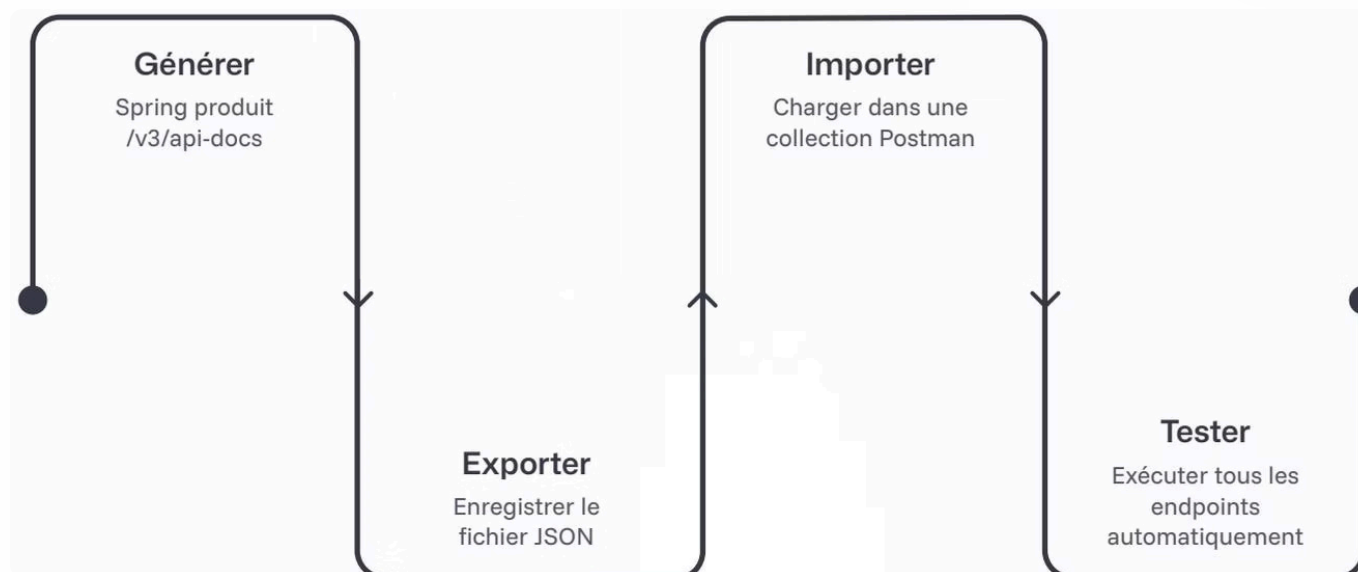
Scripts JavaScript exécutés après chaque réponse. Validation du statut, du corps, des en-têtes. Exécutables via Newman en CI/CD.



Documentation partagée

Postman génère automatiquement une documentation HTML à partir des collections, partageables avec l'équipe frontend.

Import OpenAPI dans Postman



Workflow en 4 étapes

01

Générez

Spring produit /v3/api-docs

02

Exportez

Enregistrez le fichier JSON

03

Importez

Chargez dans une collection Postman

04

Testez

Exécutez tous les endpoints
automatiquement

 Cette approche encourage le **Contract-First Development** : la spec OpenAPI est rédigée avant l'implémentation, servant de contrat validé entre équipes.

Récapitulatif du module

Architecture REST

- Contraintes REST et niveaux Richardson
- Cycle de vie d'une requête Spring MVC
- Conception des URI et content negotiation
- Codes de statut HTTP et sémantique

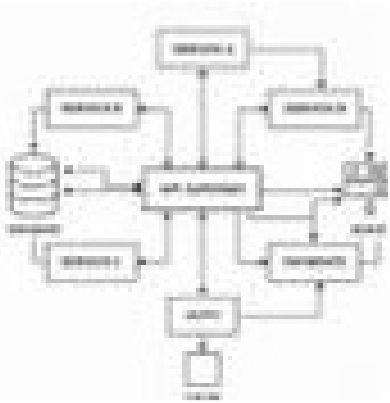
Implémentation

- @RestController, annotations de mapping
- @PathVariable, @RequestParam, @RequestBody
- ResponseEntity, Jackson, DTOs
- Bean Validation et @ControllerAdvice

Documentation

- springdoc-openapi et Swagger UI
- @Operation, @ApiResponse, @Schema
- Configuration globale et sécurité
- Stratégies de versioning d'API

Pour aller plus loin



Spring WebFlux et APIs réactives

Pour les APIs haute performance avec des dizaines de milliers de connexions concurrentes, Spring WebFlux et Project Reactor offrent une scalabilité sans égale.



Spring Security et OAuth2/OIDC

Sécurisation avancée avec OAuth2, OpenID Connect, JWT, et intégration avec Keycloak, Okta ou Azure AD.



Spring Cloud Gateway et microservices

Déploiement dans une architecture microservices : service discovery, API Gateway, circuit breaker Resilience4j, distributed tracing.